

# SQL Server Performance Masterclass

#MOC #sql-server #performance

A working collection of performance tuning techniques, learned through real-world refactoring of production stored procedures across the MFCs.

---

## The Diagnostic Toolkit

Start here. Before you can fix anything, you need to find the worst offenders and understand why they are slow.

- **Query Store Triage**: rank every query by total I/O impact. Find plan instability. Detect forced plan failures. This is the scoreboard.
  - **Density Vector**: measure data skew to decide whether density-based estimation is safe. Includes the skew analysis query and bucket histogram.
- 

## Query Plan Quality

These techniques address how the optimizer chooses its execution strategy. A wrong plan choice can mean 100x more I/O.

- **Parameter Sniffing**: the optimizer sniffs first-caller values and caches plans optimized for those values. When data is skewed, subsequent callers suffer. The fix is local variable assignment.
  - **Table Variables vs Temp Tables**: table variables always estimate 1 row. Temp tables get real statistics. For anything beyond trivial row counts, use temp tables.
  - **Non-SARGable Predicates**: function-wrapped columns prevent index seeks. The optimizer is forced to scan. Know the patterns, fix the ones you can, and document the ones you can't.
- 

## Query Structure Refactoring

These are the code-level anti-patterns that produce expensive plans. Each has a specific refactoring pattern.

- **Correlated Subqueries to CTEs**: per-row subquery execution replaced with single-pass CTEs. The single most common big-win refactoring.
  - **Scalar UDF Parallelism Barrier**: scalar functions force serial execution of the entire query. Inline them with `STRING_AGG` and CTEs to restore parallelism.
  - **LEFT JOIN OR Anti-Pattern**: LEFT JOINS with OR across different join paths prevent efficient index use. Two viable refactors: split into UNION branches, or materialize candidates into a temp table and probe each path with EXISTS.
- 

## Schema and Hint Awareness

Understanding the database objects and hints you encounter during refactoring.

- **Index Key Columns vs Included Columns**: how to read a nonclustered index, when to use INCLUDE versus a key, how the optimizer's index choice can dominate the win.
- **UNION ALL Views**: views over UNION ALL can hide complexity and amplify other anti-patterns. Know when to look through the view to the base tables.

- **FORCESEEK Hints**: a tell that someone tried to fix a scan. Investigate the root cause before removing or keeping the hint.
- **NOLOCK Strategy**: consistent dirty-read strategy for read-only display procedures. Apply uniformly. Don't mix locked and unlocked reads.

## SQL Semantics

Subtle behaviors that can bite you during refactoring if you are not careful.

- **TOP with ORDER BY Semantics**: ORDER BY on SELECT INTO is meaningless, unless TOP is present, in which case it determines which rows are selected.
- **Ambiguous Self-Comparison Predicates**: a column name that exists in both inner and outer scope resolves to the inner one. Where `Col = Col` inside a subquery is silently always true. A correctness bug, not a performance bug, but it lurks inside performance refactors.
- **STRING\_SPLIT vs WHILE Loop CSV Parsing**: the canonical replacement for the legacy WHILE-loop CSV parser. Set-based, idiomatic, and pairs naturally with a #temp table for downstream filtering.

## The Refactoring Workflow

A repeatable process for systematic performance improvement:

1. Triage. Run the **Query Store Triage** diagnostic to rank offenders by total I/O.
2. Profile. For top offenders, check for plan instability (**Parameter Sniffing**) and run skew analysis (**Density Vector**).
3. Read the code. Look for the anti-patterns: correlated subqueries, scalar UDFs, LEFT JOIN + OR, table variables, non-SARGable predicates.
4. Refactor. Apply the appropriate technique. Preserve existing comments and structure.
5. Test. SET STATISTICS IO ON and SET STATISTICS TIME ON for side-by-side comparison.
6. Document. Changelog entry with before/after code, rationale, and what was not changed.
7. Verify. Confirm reads dropped. Check that elapsed is below CPU if parallelism was restored.
8. Re-triage. Run the Query Store diagnostic again. Pick the next target.

## Procedures Refactored

| Procedure                        | Version | Key Techniques Applied                                                                                                                              |
|----------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Isp_DbDeleteOldInvAuditEvents    | v1→v2   | Batch deletion, loop optimization                                                                                                                   |
| Isp_DbDeleteOldEvents            | v2→v3   | Batch deletion, loop optimization                                                                                                                   |
| Isp_DbDeleteOldPmssData          | v1→v2   | Batch deletion, loop optimization                                                                                                                   |
| Isp_DbDeleteOldWorkflowImages    | v3→v4   | Batch deletion, loop optimization                                                                                                                   |
| Isp_SrtGetShipTotelfExists       | v26→v27 | Collapsed double-query, UNION→OR EXISTS, removed dead code                                                                                          |
| Isp_RxfGetListOfManualFillGroups | v48→v49 | <b>Correlated Subqueries to CTEs, Scalar UDF Parallelism Barrier, Table Variables vs Temp Tables, Parameter Sniffing, NOLOCK Strategy</b>           |
| Isp_ImgGetListOfTopXImagesToMove | v5→v6   | <b>LEFT JOIN OR Anti-Pattern (Fix #2: Temp Table + EXISTS), Index Key Columns vs Included Columns, UNION ALL Views, TOP with ORDER BY Semantics</b> |

---

## Key Metrics

lsp\_RxfGetListOfManualFillGroups: 62,707 → ~20,910 avg logical reads (67% reduction)  
lsp\_ImgGetListOfTopXImagesToMove: 1,141,351 → ~214,200 avg logical reads (75% reduction; LOB reads eliminated)

---

*Built during live refactoring sessions, March-May 2026.*